

Investor KPI Assistant — Design and review packet

This PDF is the written submission for the agent-platform exercise. The working application is in the accompanying source folder. No candidate name appears on these materials. KPI figures come from the provided 2,000-row sample CSV loaded into PostgreSQL; there is no separate Excel model because the assignment is a software system, not a spreadsheet forecast.

1. Product

Time-constrained public-investor customers ask natural-language questions about YipitData-style KPI estimates (historical fiscal quarters and dated quarter-to-date snapshots). A React chat UI talks to a FastAPI orchestrator. The orchestrator is an MCP client: a configured LLM may call only the read-only tools on a Python MCP server. Those tools query a shared service over PostgreSQL. An external client (Cursor, Claude Desktop, or FastMCP) can attach to the same MCP server without going through the chat UI. Login was excluded by the assignment and is not built.

2. Architecture



The important split: SQL lives only in KpiService. MCP tools are a thin, logged façade. The chat API does not query the database. That is the security boundary between the model and the data layer.

3. How to run (Windows first)

Prerequisites: Git, Python 3.12+, Node.js 18+ / npm, Docker Desktop (or local PostgreSQL 16 with user/password/database kpi/kpi/kpi on port 5432), and an OpenAI API key for live chat. Three terminals must stay open.

```

docker compose up -d postgres
python -m venv backend\.venv
backend\.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
copy .env.example .env      # set OPENAI_API_KEY, never commit .env
cd backend
python -m app.seed          # must be app.seed, from backend\
  
```

```

# three windows, venv activated in each:
python -m app.mcp_server --transport http    # :8001/mcp
python -m app.main                          # :8000
cd ../frontend && npm install && npm run dev # :5173
  
```

Open <http://127.0.0.1:5173/> (prefer 127.0.0.1 over localhost on Windows)

Port	Process
5432	PostgreSQL
8001	MCP server (Streamable HTTP /mcp)
8000	FastAPI chat API
5173	Vite React UI

If OPENAI_API_KEY is missing, the UI shows a setup error and does not invent KPI values. Tests use a mocked LLM and still exercise real MCP tool calls in-process.

4. Connecting an external MCP client

HTTP: `http://127.0.0.1:8001/mcp` while the MCP process is running. stdio: `python -m app.mcp_server --transport stdio` with `cwd` set to `backend` and `DATABASE_URL` pointing at the same Postgres. A helper client is `python -m app.demo_mcp_client` (HTTP) or `--in-process`.

5. Data rules (the “calculation” layer)

All numeric work is in the seeded table, not a workbook. The sample has 2,000 rows: 20 companies × 5 KPIs × 16 historical quarters (2022Q1–2025Q4) plus four 2026Q1 QTD snapshots (as_of 2026-01-31, 2026-02-15, 2026-02-28, 2026-03-15).

- Values stored as NUMERIC(18, 2) with the CSV unit (\$, \$MM, subs, units).
- Uniqueness: (ticker, kpi, period, estimate_type, as_of) with NULLS NOT DISTINCT.
- Historical rows have as_of NULL (completed quarter). QTD rows always have as_of.
- Latest QTD = max(as_of) per company/KPI/period. In this sample that is 2026-03-15, not “current” vs today.
- QTD is intra-quarter. The system does not treat it as a finished quarter, does not infer a subscriber growth rate from net-added subscribers, and does not extrapolate a full-quarter value unless that math is explicit and labeled.
- Worked example (source CSV): Imaginary Streaming Company (IGC), Total Revenue (\$MM), 2026Q1 QTD as of 2026-03-15 = 627.45. Earlier snapshots: 263.09 (2026-01-31), 395.67 (2026-02-15), 503.52 (2026-02-28). 2022Q1 historical = 519.63 (no as_of).

6. MCP tools

Tool	When the model should call it
search_catalog	Discover sectors, companies, tickers, KPI names
get_company_estimates	History plus the latest QTD snapshot for a ticker
get_qtd_snapshots	Earlier dated QTD prints / intra-quarter drill-down

Unknown tickers, misspelled KPIs, bad periods, and unknown sectors return structured {ok: false, error, message, suggestions} so the model can recover. “Subscribers” is ambiguous and is not auto-resolved to one KPI.

7. Design decisions and trade-offs

- Three tools, not a REST mirror: discover, fetch the working set, drill into QTD.
- Shared KpiService behind MCP so chat never imports SQL.
- Recoverable tool errors instead of thrown ToolError, which looks like a crash to an agent.
- Latest QTD is max(as_of), never CURRENT_DATE.
- Chat is a real MCP client. Missing API key does not fall back to scripted answers.
- Loop guards: MAX_TOOL_ROUNDS=6 and MAX_TOOL_CALLS=12.
- No login (assignment exclusion). Production tenancy is described, not faked.
- Trade-off: HTTP MCP is easy to attach; stdio is still provided for desktop hosts. Result size is bounded with limit.

8. Tenancy, security, monitoring (not built)

Today every row is global. Production would add tenant_id or product entitlement from a verified credential before tool dispatch, and Postgres RLS so the MCP process cannot read another tenant even if a prompt injects a ticker. The LLM must never pass tenant_id. The model only sees tool JSON — never DATABASE_URL or OPENAI_API_KEY. MCP stays read-only. Each tool log line has tool, redacted arguments, outcome, duration_ms, and correlation_id. The UI trace is the human-facing subset. Production would add an audit store, OpenTelemetry across chat → MCP → SQL, and alerts on error rate, loop-limit hits, and P95 latency.

9. Future improvements

- Stream tokens and tool events over SSE (perceived latency).
- Entitlements for which products/KPIs a customer may see.
- Materialized latest-QTD view if snapshots grow past daily.
- KPI aliases in the database rather than Python.
- Eval set of investor questions with golden tool traces.
- Rate limits and per-tenant quotas on MCP.

10. AI use versus deliberate decisions

AI was used as a coding assistant: scaffolding FastAPI/Vite files, drafting tests, and shaping README prose. Installed library APIs were checked rather than guessed (FastMCP 4 Client.call_tool, OpenAI 3.x nested tools[].function, SQLAlchemy postgresql_nulls_not_distinct). The engineering exercise recommended this and asked for this section. This packet does not claim that AI was unused.

Deliberate choices: tool schemas; latest QTD is not “today”; structured recoverable errors; KpiService as the only SQL owner; chat as an MCP client with loop caps and no silent LLM fallback; uniqueness and historical/QTD check constraints; log redaction without putting secrets in tool arguments.

11. Limits and verification

- 24 automated tests passed (CSV import, latest QTD, history, suggestions, MCP tools, mocked LLM chat flow).
- Independent MCP client listed tools and returned IGC 627.45 \$MM for 2026Q1 as of 2026-03-15.
- Without an API key, chat returns 503 llm_not_configured and does not fabricate data.
- Chat wait time is dominated by sequential OpenAI rounds, not Postgres. The UI shows model vs tool timings.
- Sample coverage is 20 fictional companies and five KPIs.

See RUNBOOK.pdf for the full local-run and troubleshooting text. See DATA-NOTES.pdf for the calculation/source-of-truth note requested alongside code.